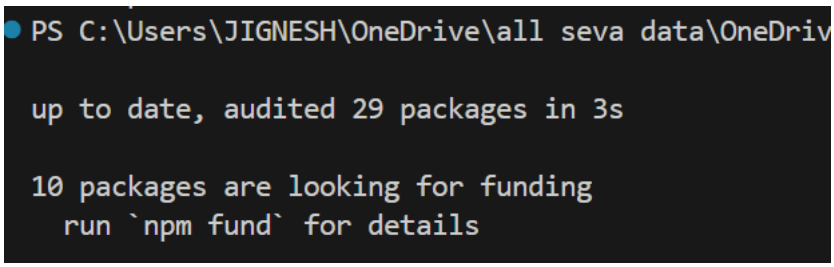
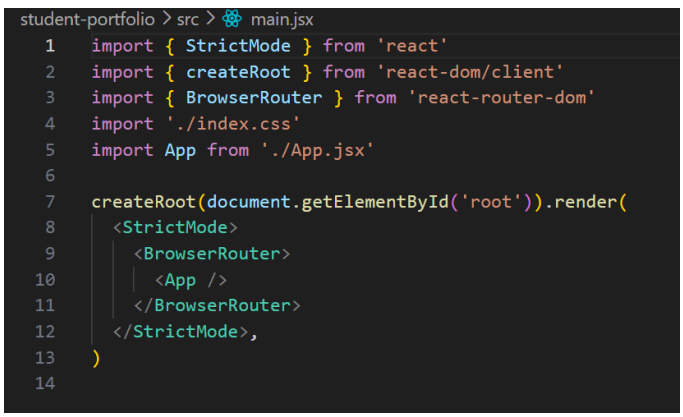
Practical 2: State Management and Routing in React	
Objective	To implement reactive state management using useState and multi-page navigation using React Router in the existing portfolio application.
Architecture / Diagram	
Diagram	App.jsx (wrapped in <BrowserRouter>) ├── NavBar.jsx (links to all routes) ├── Route: "/" → Home.jsx ├── Route: "/projects" → Projects.jsx └── Route: "/contact" → Contact.jsx (controlled form, useState)
Problem Definition	
Problem Statement	• Extend the portfolio application from Practical 1 by adding React Router. • Implement a navigation bar with at least 3 routes: Home, Projects, and Contact. • Each route must render a distinct component without a full page reload. • Add at least 2 useState variables used meaningfully one for toggling UI visibility and one for managing a form input on the Contact page. • The Contact page must include a controlled input that captures and displays user input in real time.
Steps	1. Install React Router in the existing portfolio project: npm install react-router-dom  2. Wrap the App component with BrowserRouter in main.jsx: import { BrowserRouter } from 'react-router-dom'; <BrowserRouter> <App /> </BrowserRouter> 

3. Define 3 routes inside App.jsx using Routes and Route:

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/projects" element={<Projects />} />
  <Route path="/contact" element={<Contact />} />
</Routes>
```

4. Build the NavBar using Link (not <a> tags) to avoid full page reloads.

```jsx
<div className="app">
  <NavBar darkMode={darkMode} toggleDarkMode={toggleDarkMode} />
  <Routes>
    <Route path="/" element={<Home darkMode={darkMode} />} />
    <Route path="/projects" element={<Projects darkMode={darkMode} />} />
    <Route path="/contact" element={<Contact darkMode={darkMode} />} />
    <Route path="*" element={<NotFound darkMode={darkMode} />} />
  </Routes>
  <Footer darkMode={darkMode} />
</div>
```

5. Inside Contact.jsx, create a controlled input tied to useState:

```
const [message, setMessage] = useState('');
<input value={message} onChange={(e) => setMessage(e.target.value)} />
```

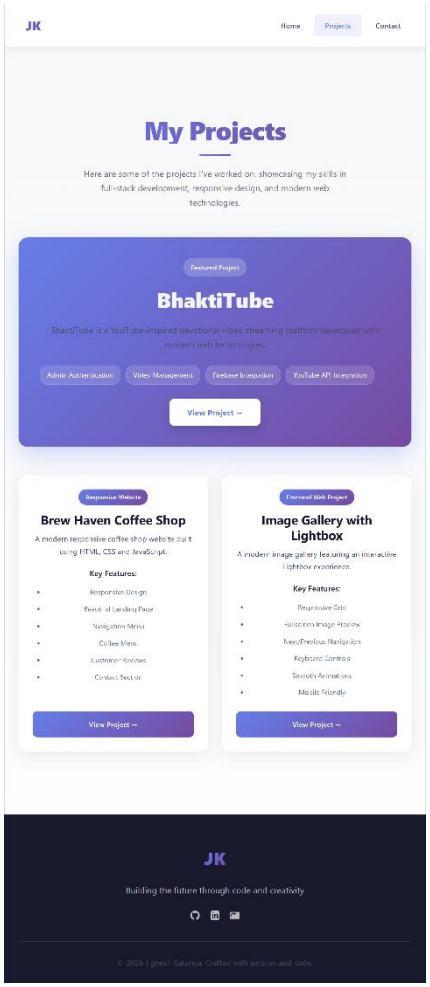
```jsx
import { useState } from 'react'

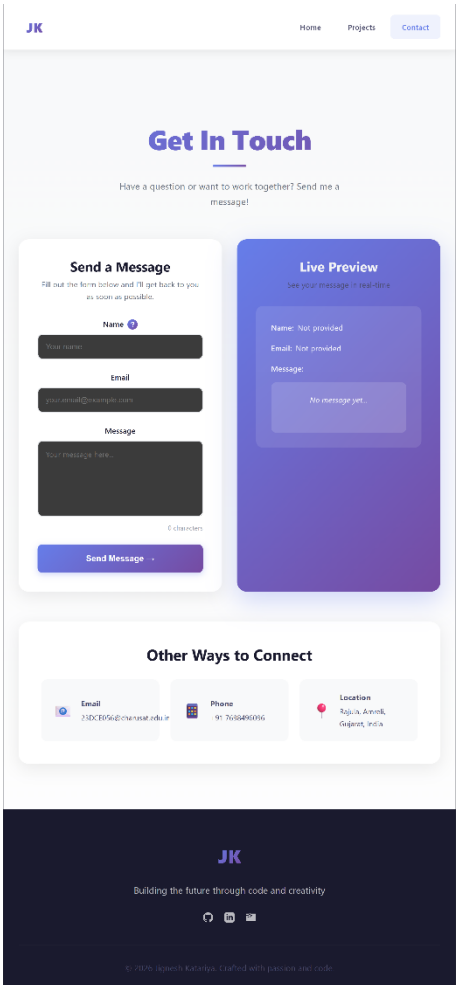
function Contact({ darkMode }) {
  // useState for form inputs
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    message: ''
  })
```

6. Add a second useState variable to toggle visibility of an element (e.g., a help tooltip).

```jsx
const [showHelp, setShowHelp] = useState(false)
```

Output





Key Questions / Analysis

1. **What is the role of each component in the overall UI structure?**

Each component is responsible for a specific part of the UI, making the application modular, organized, and easy to maintain.

2. **How does React re-render when props change?**

When props change, React updates the Virtual DOM, compares it with the previous version, and re-renders only the affected components.

3. **Why is component reusability important in large-scale applications?**

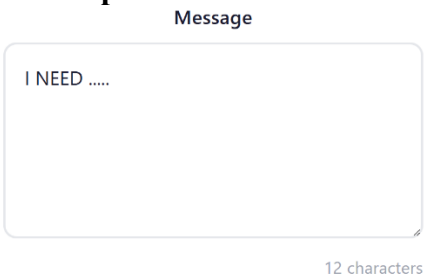
Reusable components reduce code duplication, improve consistency, simplify maintenance, and speed up development.

Supplementary Problems

1. **Add a 404 Not Found route that renders a custom error component for unknown paths.**

```jsx
<div style={dynamicPageStyle}>
  <div style={containerStyle}>
    <h1 style={errorCodeStyle}>404</h1>
    <h2 style={dynamicHeadingStyle}>Page Not Found</h2>
    <p style={dynamicMessageStyle}>
      Oops! The page you're looking for doesn't exist or has been moved.
    </p>
    <Link to="/" style={dynamicLinkStyle}>
      Go Back Home
    </Link>
  </div>
</div>
```

2. **Store the contact form input in state and display a live character count below the input.**



3. **Implement a dark/light mode toggle using useState that applies a CSS class to the root element.**



Key Skills Addressed

1. Installing and configuring React Router v6
2. Defining and navigating between routes without page reload
3. Managing component state using the useState hook
4. Building controlled form inputs tied to state

Learning Outcome	Students will be able to implement client-side routing with React Router and manage dynamic UI state using useState, resulting in a multi-page application without full page reloads.